

■ DSA Mastery Guide

Crack 6–10 LPA Tech Companies

What's inside:

- All key DSA topics with patterns & approach
- Must-know LeetCode problems per topic
- Time & Space complexity quick reference
- Interview tips & common mistakes to avoid
- 8-week study plan

■ **Tip:** Focus on understanding PATTERNS, not memorising solutions. One pattern unlocks 10+ problems.

■ Complexity Cheat Sheet

Data Structure / Algorithm	Time Complexity	Space
Array access / update	$O(1)$	$O(1)$
Array search (unsorted)	$O(n)$	$O(1)$
Binary Search	$O(\log n)$	$O(1)$
Hash Map get/put	$O(1)$ avg	$O(n)$
Stack / Queue push/pop	$O(1)$	$O(n)$
Heap insert / extract	$O(\log n)$	$O(n)$
BFS / DFS	$O(V + E)$	$O(V)$
Merge Sort	$O(n \log n)$	$O(n)$
Quick Sort (avg)	$O(n \log n)$	$O(\log n)$
DP (typical 2D)	$O(n*m)$	$O(n*m)$

1■■ Arrays & Strings

Key Patterns

- **Two Pointers:** Use when array is sorted or you need pairs. Move left/right inward.
- **Sliding Window:** For subarray/substring problems with a constraint (size k, sum, unique chars).
- **Prefix Sum:** Precompute cumulative sum to answer range queries in $O(1)$.
- **Kadane's Algorithm:** Max subarray sum — track current sum, reset when negative.

Must-Know Problems

Problem	Pattern	Difficulty
Two Sum	HashMap	Easy
Best Time to Buy Stock	Single pass	Easy
Maximum Subarray	Kadane's	Medium
Longest Substring Without Repeat	Sliding Window	Medium
Container With Most Water	Two Pointers	Medium
Product of Array Except Self	Prefix/Suffix	Medium
Subarray Sum Equals K	Prefix Sum + HashMap	Medium
Minimum Window Substring	Sliding Window	Medium

■ **Tip:** For sliding window: expand right pointer, shrink left when constraint is violated.

2■■ Hashing

When to use HashMap vs HashSet

- **HashMap:** When you need key $\rightarrow$ value mapping (frequency, index storage).
- **HashSet:** When you only need to check existence / uniqueness.

Must-Know Problems

Problem	Approach
Group Anagrams	Sort each word as key in map
Top K Frequent Elements	HashMap + Heap
Longest Consecutive Sequence	HashSet — check sequence start
Valid Anagram	Frequency count array/map
Four Sum II	HashMap of pair sums

■ **Tip:** Frequency map trick: use array of size 26 for lowercase English letters — faster than HashMap.

3■■ Binary Search

Template (use this always)

```
def binary_search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Must-Know Problems

Problem	Key Idea
Search in Rotated Sorted Array	Find which half is sorted
Find Minimum in Rotated Array	Binary search on answer
Koko Eating Bananas	Binary search on speed value
Median of Two Sorted Arrays	Partition-based binary search
Search a 2D Matrix	Treat 2D as 1D index

■ **Tip:** 'Binary search on answer' pattern: when you can define a check(mid) function that is monotone (T,T,T,F,F or F,F,T,T), binary search on the value space.

4■■ Linked List

Core Techniques

- **Fast & Slow Pointers (Floyd's):** Detect cycle, find middle, find kth from end.
- **Dummy Node:** Simplifies edge cases at head during reversal/merge.
- **Reverse in-place:** Three pointers — prev, curr, next.

Problem	Technique
Reverse Linked List	Iterative 3-pointer
Linked List Cycle	Floyd's slow/fast
Merge Two Sorted Lists	Dummy node + compare
Remove Nth from End	Two pointers gap = n
Reorder List	Find mid + reverse + merge
Copy List with Random Pointer	HashMap old→new node

5■■ Stack & Queue

Monotonic Stack Pattern

Use when you need **next greater / smaller element**. Maintain a stack of decreasing or increasing elements.

Problem	Stack Type
Valid Parentheses	Simple stack
Next Greater Element	Monotonic decreasing
Daily Temperatures	Monotonic decreasing
Largest Rectangle in Histogram	Monotonic increasing
Min Stack	Two stacks / tuple
Sliding Window Maximum	Deque (monotonic)

■ **Note:** Deque (double-ended queue) is key for sliding window maximum — $O(n)$ solution.

6■■ Trees (BT + BST)

Traversal Template (DFS)

```
def dfs(node):
    if not node: return
    # Preorder: process here
    dfs(node.left)
    # Inorder: process here
    dfs(node.right)
    # Postorder: process here
```

Problem	Approach
Max Depth of Binary Tree	DFS — return 1 + max(left, right)

Diameter of Binary Tree	DFS — track max left+right height
Invert Binary Tree	DFS — swap left and right
Lowest Common Ancestor	DFS — check if both sides have node
Binary Tree Right Side View	BFS level order — take last of each level
Validate BST	DFS with min/max bounds
Kth Smallest in BST	Inorder traversal (sorted order)
Construct BT from Pre+Inorder	Recursion — find root, split

■ **Tip:** BST inorder traversal gives elements in sorted order — very useful.

7■■■ Heap / Priority Queue

Python uses **min-heap** by default (**heapq**). For max-heap, negate values.

Problem	Heap Type
Kth Largest Element	Min-heap of size k
Top K Frequent Elements	Max-heap on frequency
Merge K Sorted Lists	Min-heap with (val, list_idx)
Find Median from Data Stream	Max-heap + Min-heap
Task Scheduler	Max-heap on frequency

```
import heapq
heap = []
heapq.heappush(heap, val)    # push
heapq.heappop(heap)         # pop min
heap[0]                      # peek min
# Max-heap: push -val, negate when reading
```

8■■■ Recursion & Backtracking

Backtracking Template

```
def backtrack(start, path):
    if condition_met:
        result.append(path[:])
        return
    for i in range(start, len(nums)):
        path.append(nums[i])
        backtrack(i + 1, path) # or i for reuse
        path.pop()           # undo choice
```

Problem	Variation
Subsets	No duplicates, each element once
Permutations	All orderings, use visited[]
Combination Sum	Can reuse elements
Word Search	Grid DFS + visited marking
Palindrome Partitioning	Check palindrome before recurse
N-Queens	Track cols, diag1, diag2 sets

9■■■ Dynamic Programming

Approach

- **Step 1:** Define state — what does `dp[i]` or `dp[i][j]` represent?
- **Step 2:** Write recurrence relation.
- **Step 3:** Identify base cases.
- **Step 4:** Fill table (bottom-up) or use memoization (top-down).

Essential Problems

Problem	State / Pattern
Climbing Stairs	$dp[i] = dp[i-1] + dp[i-2]$
House Robber	$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$
Coin Change	$dp[\text{amount}] = \text{min coins needed}$
Longest Common Subsequence	$dp[i][j]$ — 2D, match or skip
Longest Increasing Subsequence	$dp[i] = \text{max LIS ending at } i$
0/1 Knapsack	$dp[i][w]$ — include or exclude item
Edit Distance	$dp[i][j]$ — insert/delete/replace
Unique Paths	$dp[i][j] = dp[i-1][j] + dp[i][j-1]$
Partition Equal Subset Sum	Knapsack variant — boolean dp

■ **Tip:** If recursion with memoization is hard to see, draw the recursion tree first, then convert to DP table.

■ Graphs

BFS vs DFS — When to use

- **BFS:** Shortest path (unweighted), level-order, minimum steps.
- **DFS:** Connectivity, cycle detection, topological sort, path existence.

BFS Template

```
from collections import deque
def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Problem	Algorithm
Number of Islands	DFS/BFS flood fill
Clone Graph	BFS + HashMap old→new
Course Schedule (Cycle?)	DFS cycle detect / Topo sort
Course Schedule II (Order)	Topological sort (Kahn BFS)
Pacific Atlantic Water Flow	BFS from both oceans
Rotting Oranges	Multi-source BFS
Cheapest Flights K Stops	Bellman-Ford / BFS with states
Word Ladder	BFS — each word is a node

■ **Tip:** For Topological Sort (Kahn's): build in-degree map, push all 0-indegree nodes to queue, process BFS.

1■■1■■ Greedy

Greedy works when **local optimal choice leads to global optimum**. Always think: 'Can DP give a counterexample?'

Problem	Greedy Choice
Jump Game	Track max reachable index
Jump Game II	Track current/next jump boundary
Merge Intervals	Sort by start, merge overlapping
Non-overlapping Intervals	Sort by end, greedily keep
Partition Labels	Track last occurrence of each char
Gas Station	Track total and running surplus

■ 8-Week Study Plan

Week	Topics	Target Problems
Week 1	Arrays, Strings, Hashing	25 problems
Week 2	Binary Search, Two Pointers, Sliding Window	20 problems
Week 3	Linked List, Stack, Queue	20 problems
Week 4	Trees (BT + BST)	25 problems
Week 5	Heap, Backtracking	20 problems
Week 6	Dynamic Programming (1D + 2D)	25 problems
Week 7	Graphs	20 problems
Week 8	Greedy + Full Revision + Mock Interviews	30 problems

Daily Routine

- **Morning (1 hr):** Solve 2 new problems.
- **Evening (30 min):** Revisit 1 problem you struggled with.
- **Weekend:** 1 full mock interview (timed, no hints).

■ **Tip:** After solving, always check: Can I reduce space? Can I reduce time? Can I code it cleaner?

■ Interview Tips

During the Interview

- **Clarify first:** Ask about constraints — array size, duplicates, negative numbers, empty input.
- **Think aloud:** Say your brute force approach first, then optimize.
- **Dry run:** Trace your code on the example before saying you're done.
- **Edge cases:** Empty input, single element, all same elements, negative numbers.

Common Mistakes

- Off-by-one errors in binary search (use $lo + (hi-lo)//2$ to avoid overflow).
- Forgetting to handle null/empty cases in tree/linked list problems.
- Modifying input when you shouldn't — ask if in-place is allowed.
- Using $O(n)$ space when $O(1)$ is possible (two-pointer instead of extra array).

■ **Note:** Do NOT jump to code immediately. Spend 5 minutes thinking and explaining your approach first.

Complexity You Must Justify

After every solution, be ready to state: **Time:** $O(?)$ and **Space:** $O(?)$ and WHY.

Best of luck! You've got this. ■